Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий – РТФ

«Введение в алгоритмы и структуры данных. Исследование возможностей коллекций в языке программирования Java»

Лабораторное задание №3

Группа: РИМ-150950
Студент: Владимиров Н.А.
Вариант: 4

Преподаватель: Архипов Н. А.

Екатеринбург
2026

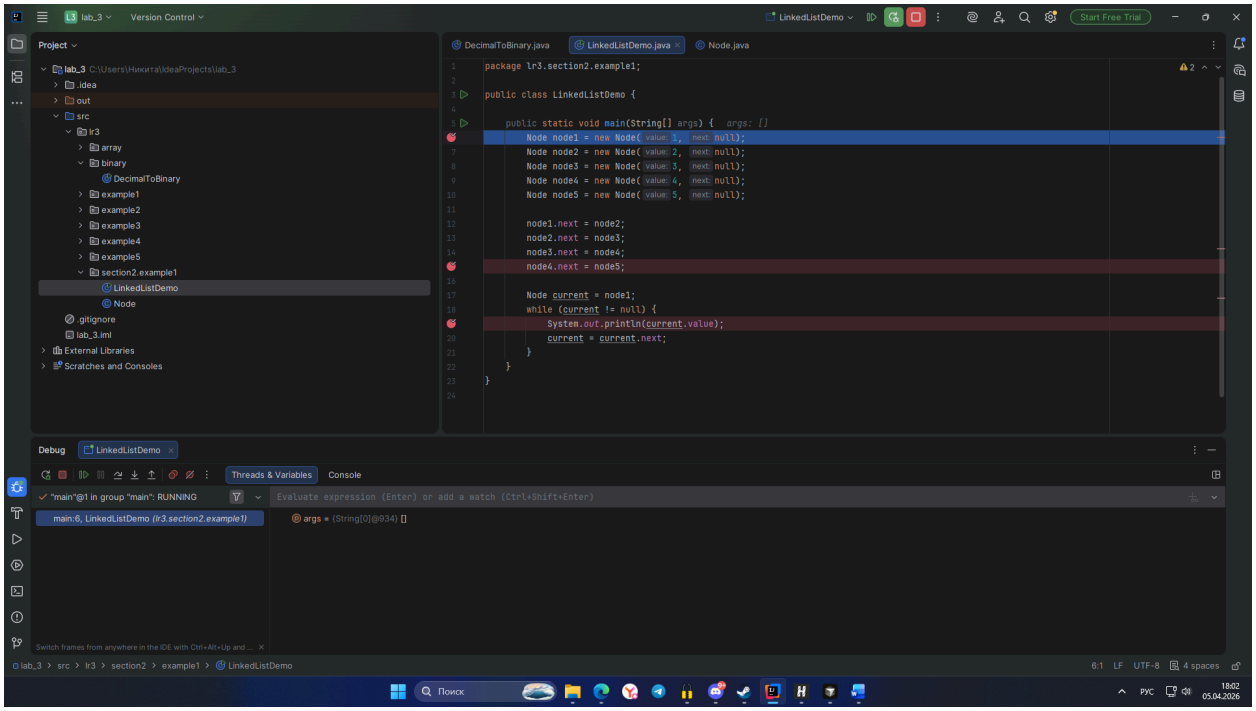
Цель работы: приобретение навыков работы с рекурсивными методами, знакомство с динамическими структурами данных, приобретение навыков создания и использования простейшей динамической структуры, исследование возможностей коллекций в языке программирования Java, сравнение коллекций по времени выполнения операций, знакомство с реализацией коллекций.

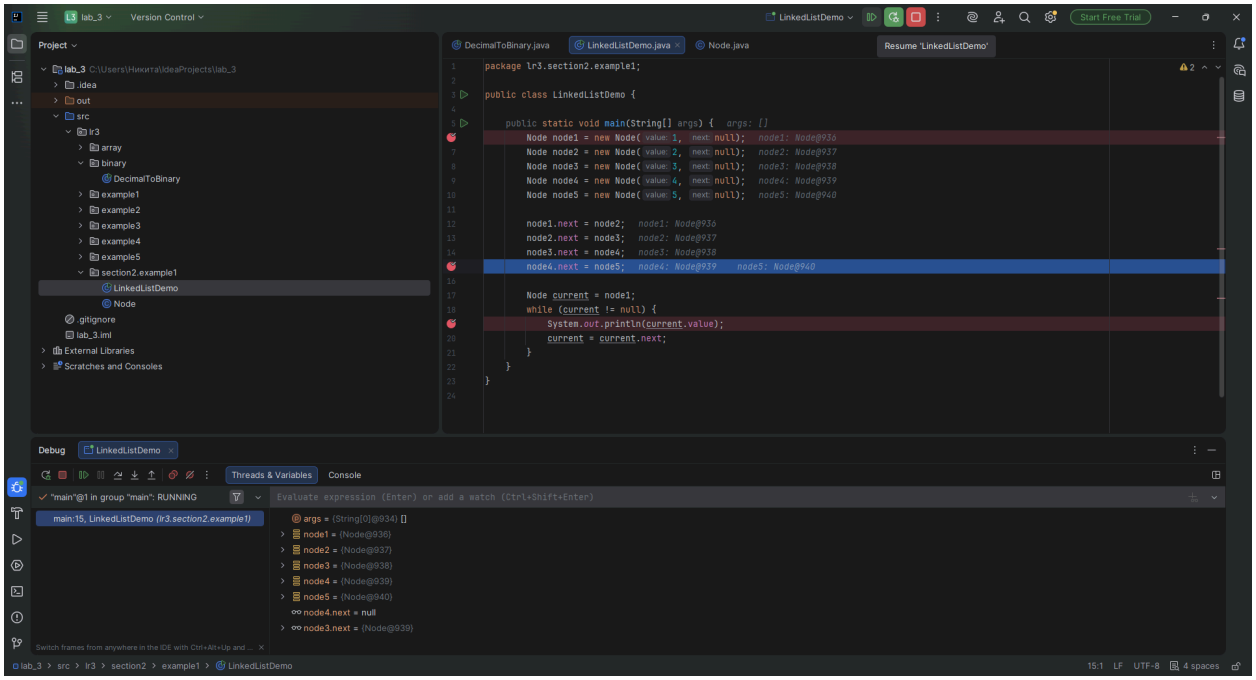
Ход работы:

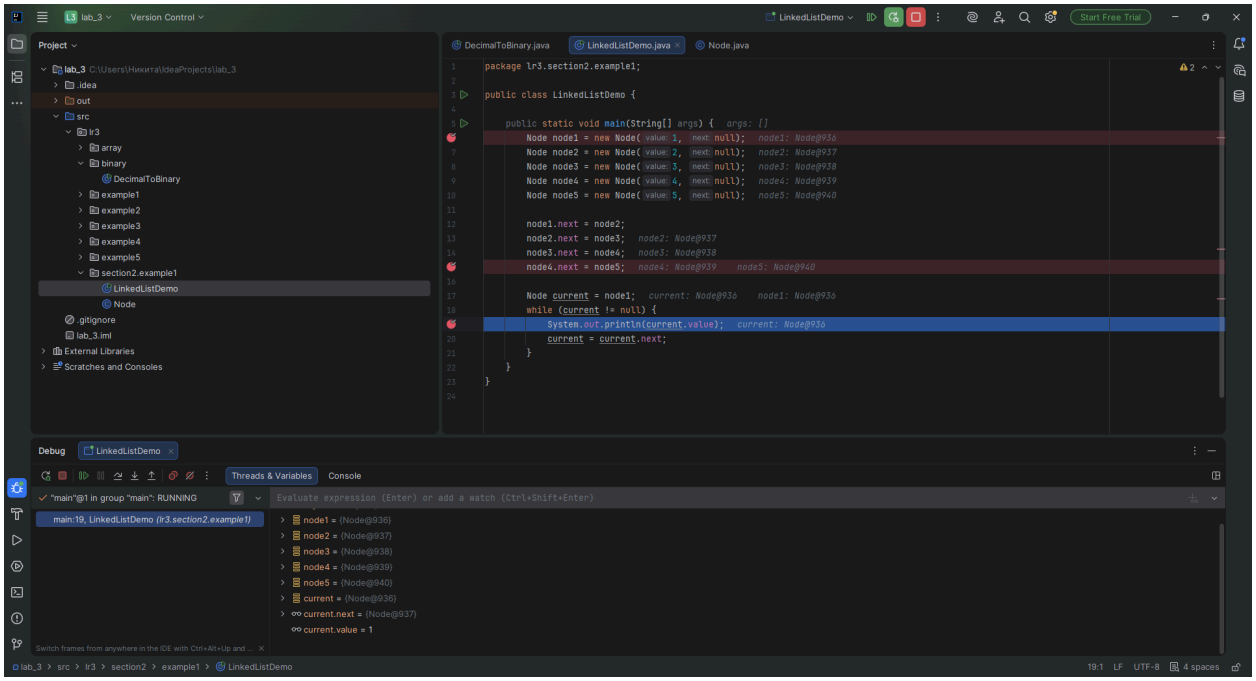
1. Выполнение задач

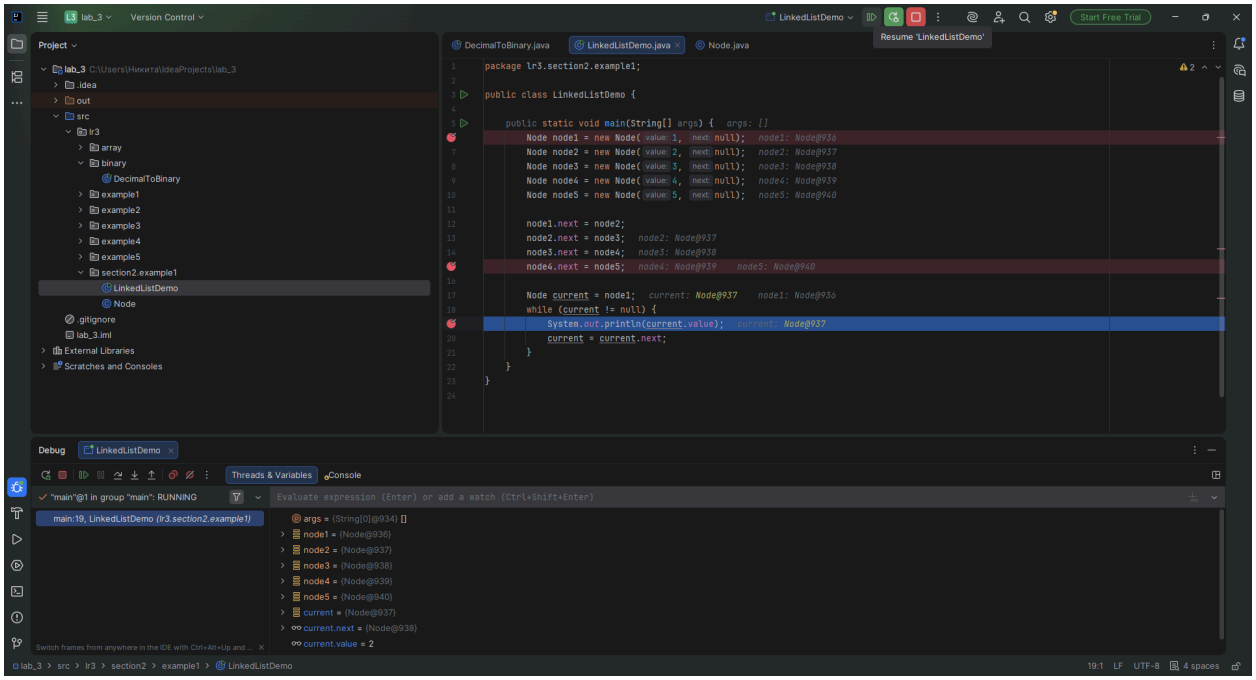
Коды задач представлены на Github [n0vision/Java_Labs_2026]

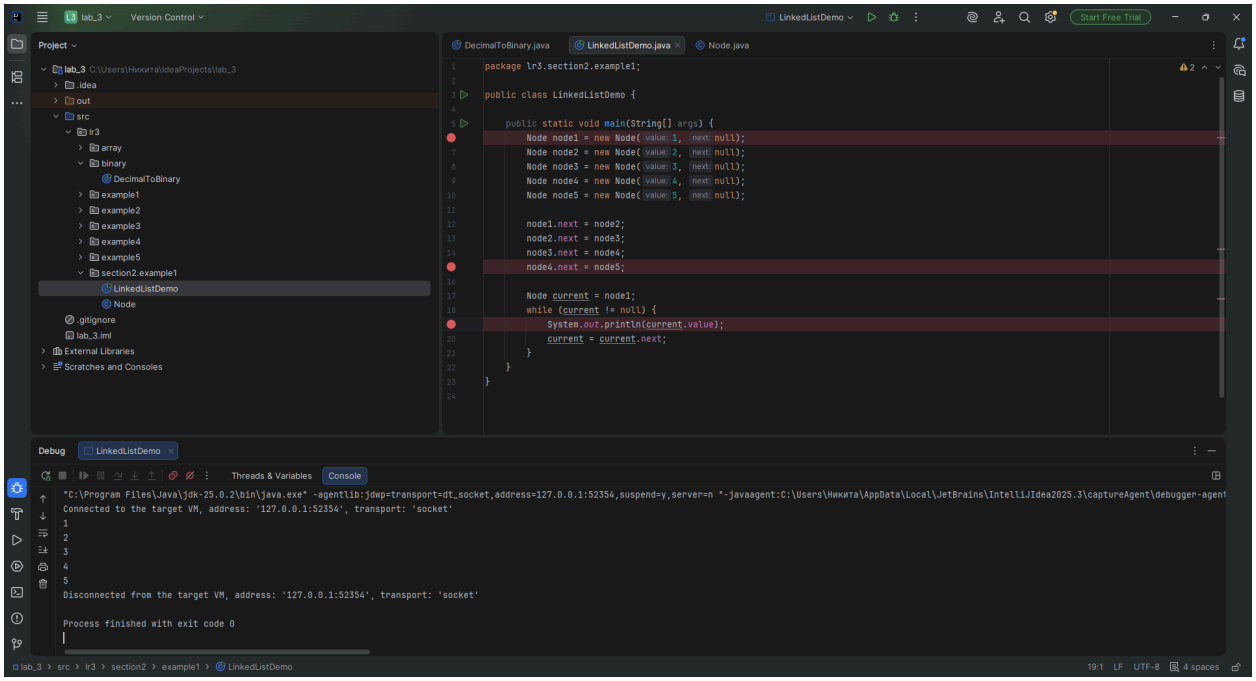
1.4 Построить однонаправленный список из предварительно созданных независимых элементов (узлов), у которых значением поля будет целое число, равное номеру элемента, и вывести значения полей на экран.




lab_3
Version Control
LinkedListDemo
Start Free Trial
Project
lab_3 C:\Users\Никита\IdeaProjects\lab_3
.idea
out
src
lr3
array
binary
DecimalToBinary
example1
example2
example3
example4
example5
section2.example1
LinkedListDemo
Node
.gitignore
lab_3.iml
External Libraries
Scratches and Consoles
DecimalToBinary.java
LinkedListDemo.java
Node.java
Resume 'LinkedListDemo'
package lr3.section2.example1;
public class LinkedListDemo {
public static void main(String[] args) { args: []
Node node1 = new Node(value: 1, next: null); node1: Node@936
Node node2 = new Node(value: 2, next: null); node2: Node@937
Node node3 = new Node(value: 3, next: null); node3: Node@938
Node node4 = new Node(value: 4, next: null); node4: Node@939
Node node5 = new Node(value: 5, next: null); node5: Node@940
node1.next = node2; node1: Node@936
node2.next = node3; node2: Node@937
node3.next = node4; node3: Node@938
node4.next = node5; node4: Node@939 node5: Node@940
Node current = node1;
while (current != null) {
System.out.println(current.value);
current = current.next;
}
}
}
Debug
LinkedListDemo
Threads & Variables
Console
"main"@1 in group "main": RUNNING
Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)
main:15, LinkedListDemo (lr3.section2.example1)
args = {String[0]@934} []
node1 = {Node@936}
node2 = {Node@937}
node3 = {Node@938}
node4 = {Node@939}
node5 = {Node@940}
node4.next = null
node3.next = {Node@939}
Switch frames from anywhere in the IDE with Ctrl+Alt+Up and ...
lab_3 > src > lr3 > section2 > example1 > LinkedListDemo
15:1 LF UTF-8 4 spaces

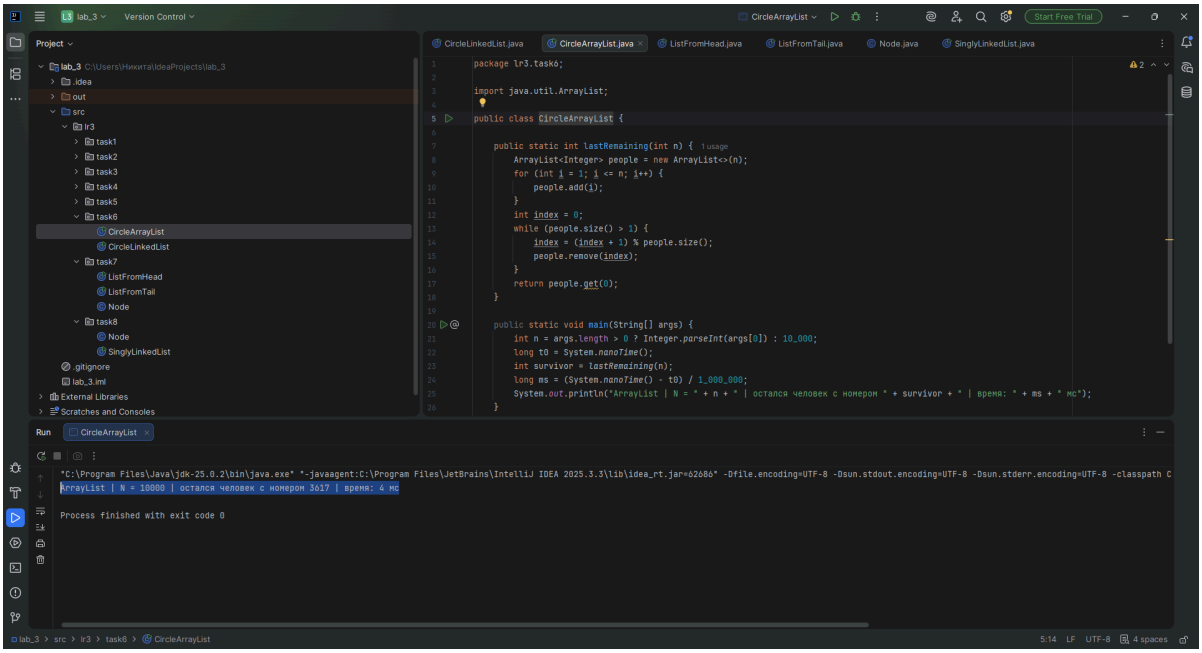

lab_3
Version Control
LinkedListDemo
Start Free Trial
Project
lab_3 C:\Users\Никита\IdeaProjects\lab_3
.idea
out
src
lr3
array
binary
DecimalToBinary
example1
example2
example3
example4
example5
section2.example1
LinkedListDemo
Node
.gitignore
lab_3.iml
External Libraries
Scratches and Consoles
DecimalToBinary.java
LinkedListDemo.java
Node.java
package lr3.section2.example1;
public class LinkedListDemo {
public static void main(String[] args) { args: []
Node node1 = new Node(value: 1, next: null); node1: Node@936
Node node2 = new Node(value: 2, next: null); node2: Node@937
Node node3 = new Node(value: 3, next: null); node3: Node@938
Node node4 = new Node(value: 4, next: null); node4: Node@939
Node node5 = new Node(value: 5, next: null); node5: Node@940
node1.next = node2;
node2.next = node3; node2: Node@937
node3.next = node4; node3: Node@938
node4.next = node5; node4: Node@939 node5: Node@940
Node current = node1; current: Node@936 node1: Node@936
while (current != null) {
System.out.println(current.value); current: Node@936
current = current.next;
}
}
}
Debug
LinkedListDemo
Threads & Variables
Console
"main"@1 in group "main": RUNNING
Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)
main:19, LinkedListDemo (lr3.section2.example1)
node1 = {Node@936}
node2 = {Node@937}
node3 = {Node@938}
node4 = {Node@939}
node5 = {Node@940}
current = {Node@936}
current.next = {Node@937}
current.value = 1
Switch frames from anywhere in the IDE with Ctrl+Alt+Up and ...
lab_3 > src > lr3 > section2 > example1 > LinkedListDemo
19:1 LF UTF-8 4 spaces

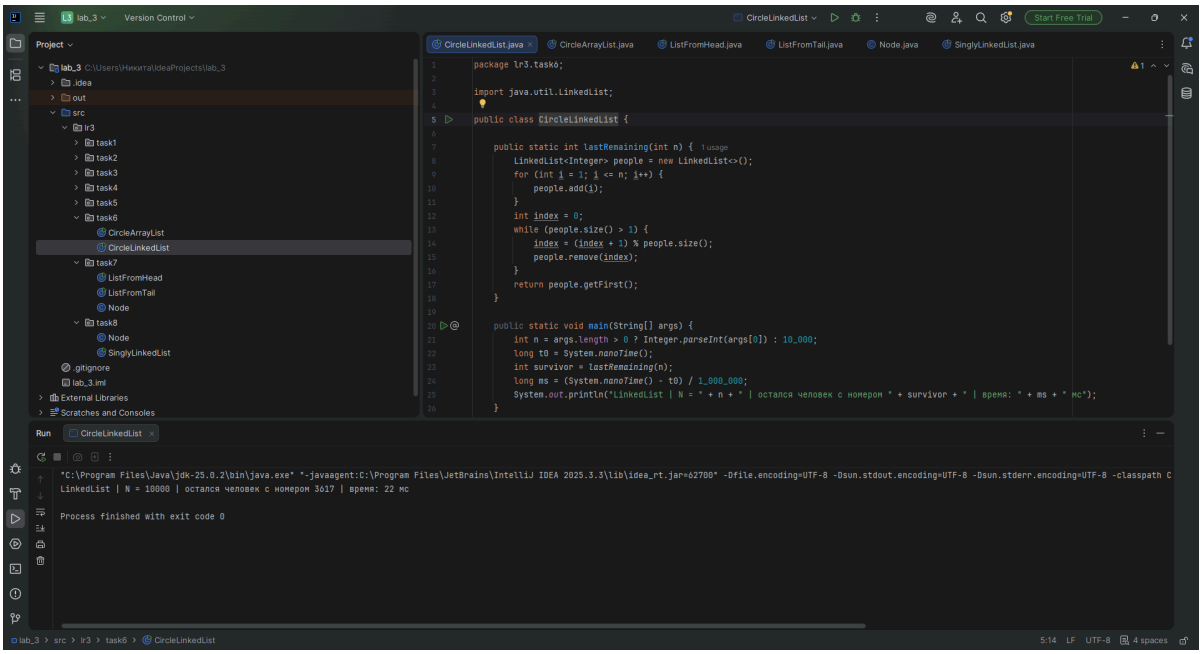




Рисунки 1-5. Результат работы отладчика

1.6 В кругу стоят N человек, пронумерованных от 1 до N. При ведении счета по кругу вычеркивается каждый второй человек, пока не останется один. Составить две программы, моделирующие процесс. Одна из программ должна использовать класс **ArrayList**, а вторая — **LinkedList**. Какая из двух программ работает быстрее? Почему?





Рисунки 6-7. Сравнение ArrayList и LinkedList по скорости исполнения

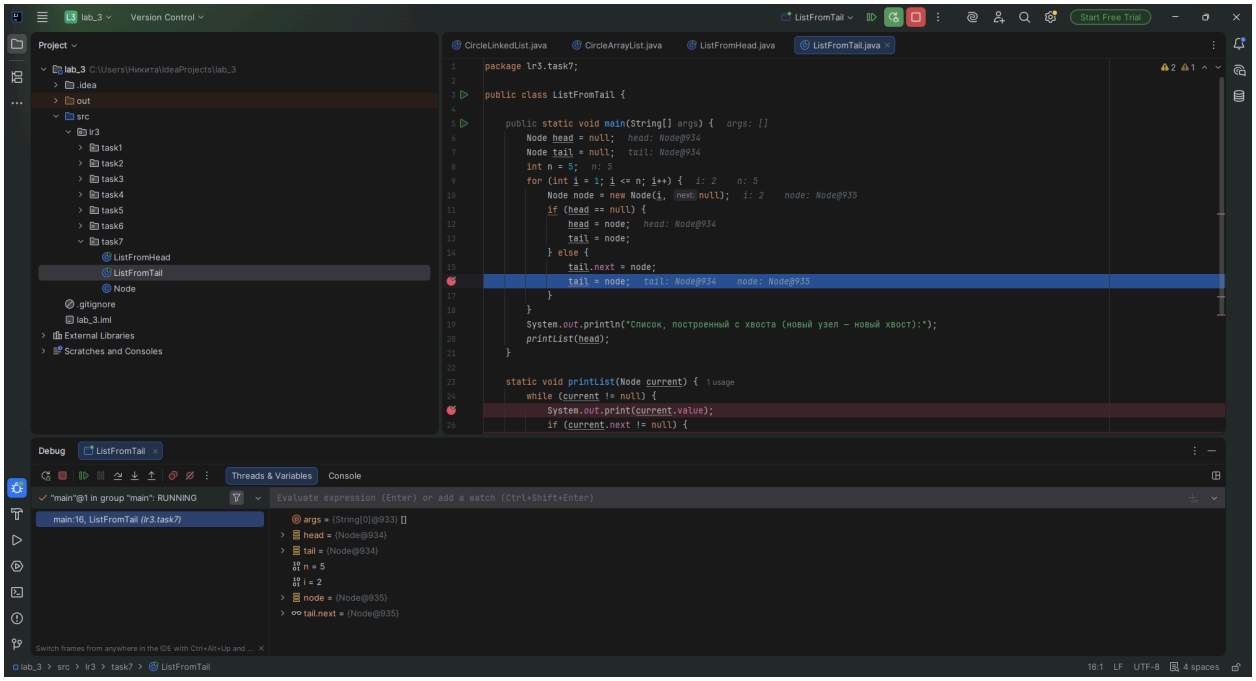
На типичном запуске быстрее вариант с ArrayList.

В обоих случаях вызывается remove(int index) по индексу. У ArrayList доступ по индексу O(1), удаление — O(n) из-за сдвига хвоста.

У LinkedList метод get(i) / поиск узла по индексу каждый раз идёт по списку от начала (или с конца), то есть O(n), плюс само удаление узла O(1) после того, как узел найден. В итоге на один шаг получается O(n), с большей константой (обход ссылок, кэш), чем у компактного массива в ArrayList.

1.7 Создать два проекта, в которых продемонстрировать два способа создания линейного однонаправленного списка (с головы и с хвоста) согласно примеру 2 из второго раздела. Отработать код с помощью отладчика и привести скриншоты минимум трех точек, обработанных отладчиком.

Tail



```java
package lr3.task7;

public class ListFromTail {

    public static void main(String[] args) {
        Node head = null;
        Node tail = null;
        int n = 5;
        for (int i = 1; i <= n; i++) {
            Node node = new Node(i, next: null);
            if (head == null) {
                head = node;
                tail = node;
            } else {
                tail.next = node;
                tail = node;
            }
        }
        System.out.println("Список, построенный с хвоста (новый узел — новый хвост):");
        printList(head);
    }

    static void printList(Node current) {  1 usage        current: Node@934
        while (current != null) {
            System.out.print(current.value);   current: Node@934
            if (current.next != null = true ) {
```

```
printList:25, ListFromTail (lr3.task7)
main:20, ListFromTail (lr3.task7)
```

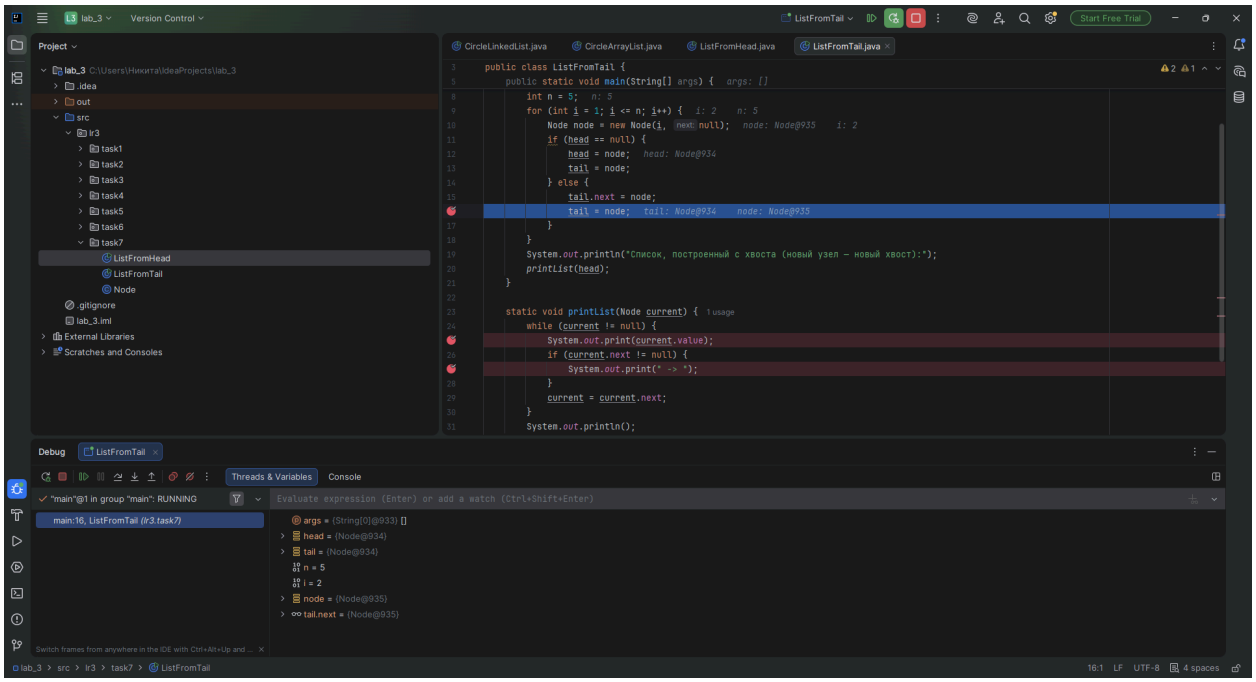
```
current = {Node@934}
current.next = {Node@935}
current.value = 1
```

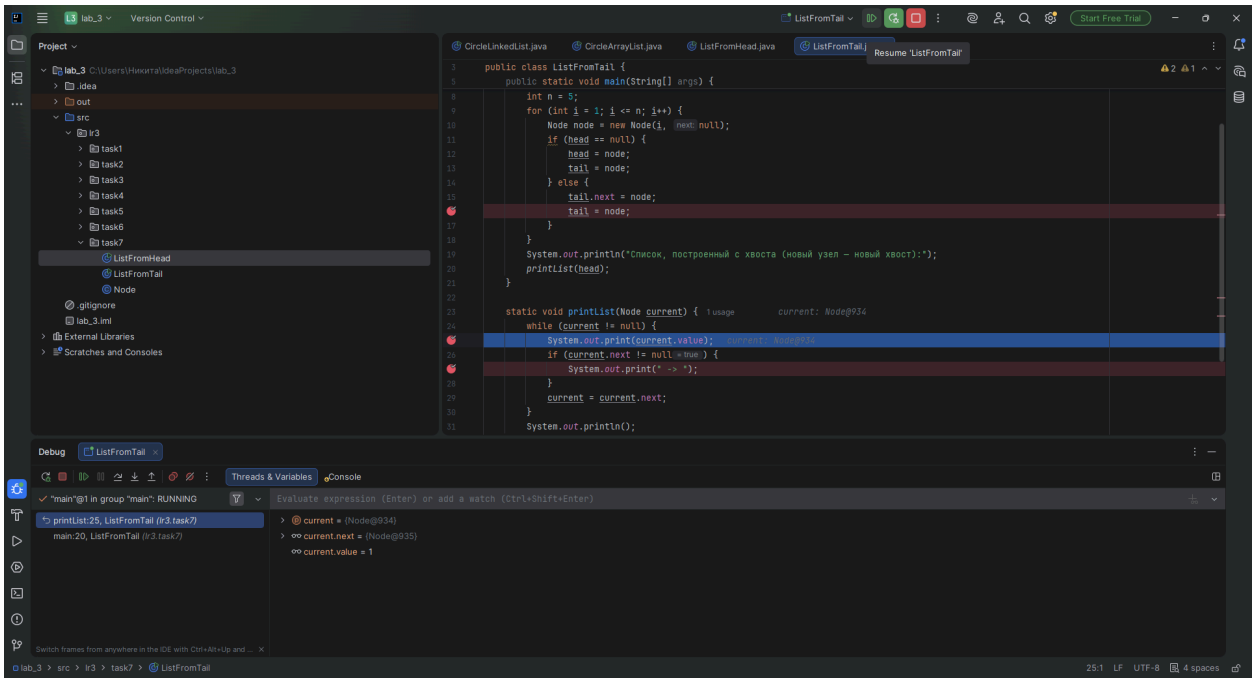
```java
public class ListFromTail {
    public static void main(String[] args) {
        int n = 5;
        for (int i = 1; i <= n; i++) {
            Node node = new Node(i, next: null);
            if (head == null) {
                head = node;
                tail = node;
            } else {
                tail.next = node;
                tail = node;
            }
        }
        System.out.println("Список, построенный с хвоста (новый узел — новый хвост):");
        printList(head);
    }

    static void printList(Node current) {  1 usage        current: Node@934
        while (current != null) {
            System.out.print(current.value);
            if (current.next != null) {   current: Node@934
                System.out.print(" -> ");
            }
            current = current.next;
        }
        System.out.println();
```

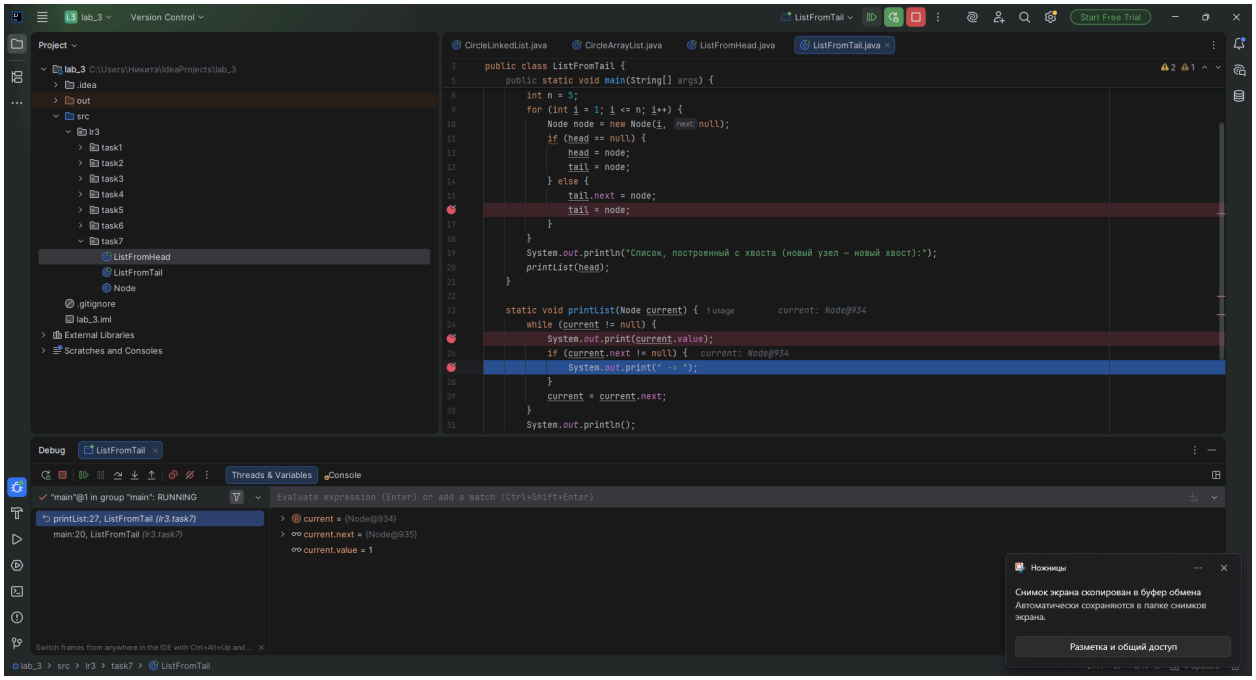
```
printList:27, ListFromTail (lr3.task7)
main:20, ListFromTail (lr3.task7)
```

```
current = {Node@934}
current.next = {Node@935}
current.value = 1
```

Head


lab_3
Version Control
ListFromTail
Start Free Trial
Project
lab_3 C:\Users\Никита\IdeaProjects\lab_3
.idea
out
src
lr3
task1
task2
task3
task4
task5
task6
task7
ListFromHead
ListFromTail
Node
.gitignore
lab_3.iml
External Libraries
Scratches and Consoles
CircleLinkedList.java
CircleArrayList.java
ListFromHead.java
ListFromTail.java
public class ListFromTail {
public static void main(String[] args) { args: []
int n = 5; n: 5
for (int i = 1; i <= n; i++) { i: 2 n: 5
Node node = new Node(i, next: null); node: Node@935 i: 2
if (head == null) {
head = node; head: Node@934
tail = node;
} else {
tail.next = node;
tail = node; tail: Node@934 node: Node@935
}
}
System.out.println("Список, построенный с хвоста (новый узел — новый хвост):");
printList(head);
}
static void printList(Node current) { 1 usage
while (current != null) {
System.out.print(current.value);
if (current.next != null) {
System.out.print(" -> ");
}
current = current.next;
}
System.out.println();
Debug
ListFromTail
Threads & Variables
Console
"main"@1 in group "main": RUNNING
Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)
main:16, ListFromTail (lr3.task7)
args = {String[0]@933} []
head = {Node@934}
tail = {Node@934}
n = 5
i = 2
node = {Node@935}
tail.next = {Node@935}
Switch frames from anywhere in the IDE with Ctrl+Alt+Up and ...
lab_3 > src > lr3 > task7 > ListFromTail
16:1 LF UTF-8 4 spaces

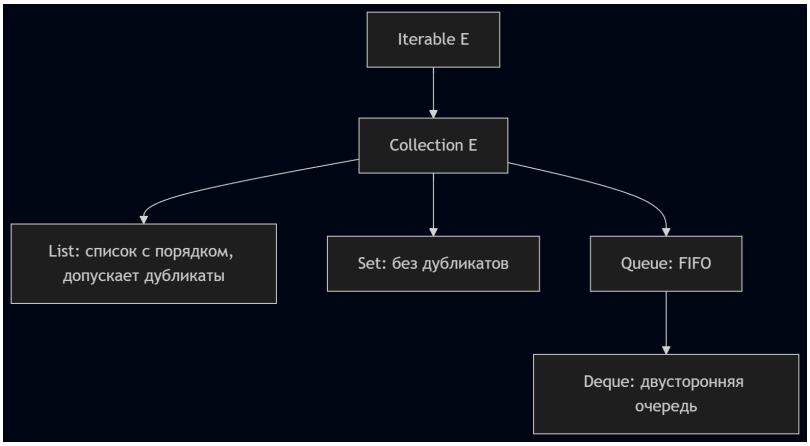

lab_3
Version Control
ListFromTail
Start Free Trial
Resume 'ListFromTail'
Project
lab_3 C:\Users\Никита\IdeaProjects\lab_3
.idea
out
src
lr3
task1
task2
task3
task4
task5
task6
task7
ListFromHead
ListFromTail
Node
.gitignore
lab_3.iml
External Libraries
Scratches and Consoles
CircleLinkedList.java
CircleArrayList.java
ListFromHead.java
public class ListFromTail {
public static void main(String[] args) {
int n = 5;
for (int i = 1; i <= n; i++) {
Node node = new Node(i, next: null);
if (head == null) {
head = node;
tail = node;
} else {
tail.next = node;
tail = node;
}
}
System.out.println("Список, построенный с хвоста (новый узел — новый хвост):");
printList(head);
}
static void printList(Node current) { 1 usage current: Node@934
while (current != null) {
System.out.print(current.value); current: Node@934
if (current.next != null = true) {
System.out.print(" -> ");
}
current = current.next;
}
System.out.println();
Debug
ListFromTail
Threads & Variables
Console
"main"@1 in group "main": RUNNING
Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)
printList:25, ListFromTail (lr3.task7)
main:20, ListFromTail (lr3.task7)
current = {Node@934}
current.next = {Node@935}
current.value = 1
Switch frames from anywhere in the IDE with Ctrl+Alt+Up and ...
lab_3 > src > lr3 > task7 > ListFromTail
25:1 LF UTF-8 4 spaces

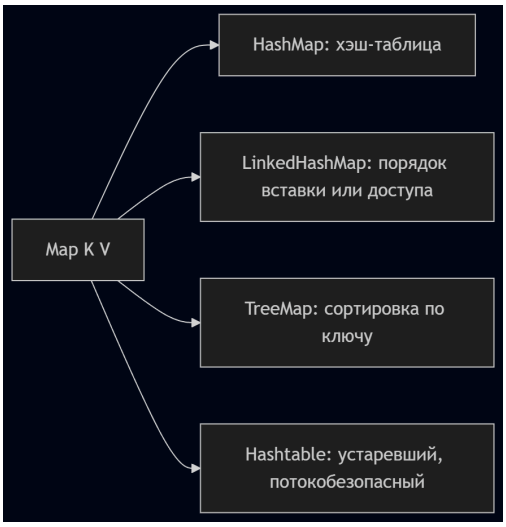


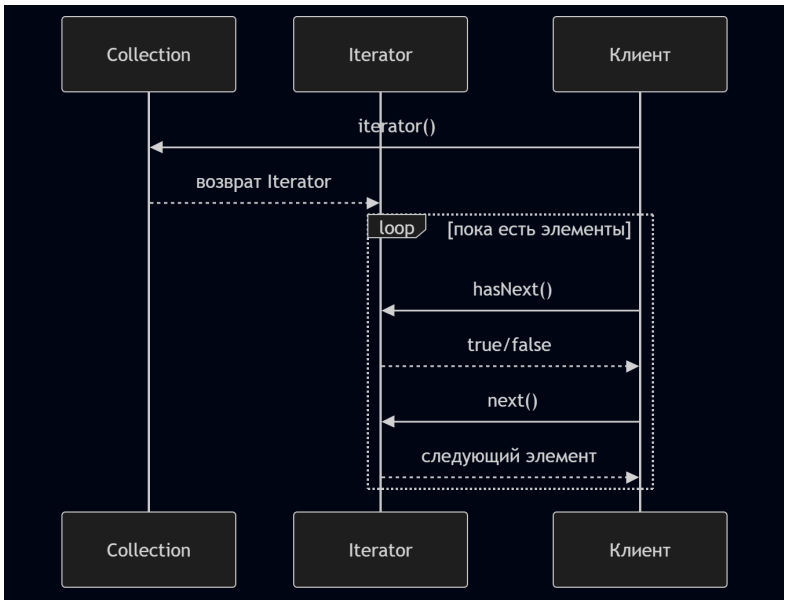
Рисунки 8-13. Результат работы отладчика

Интерфейс java.util.Collection<E> - корневой контракт для большинства коллекций, хранящих набор элементов (каждый элемент - значение). От него наследуются List, Set, Queue и др. Коллекция задаёт общие операции: добавление и удаление элементов, проверка принадлежности, размер, итератор, массовые операции. Сам Collection расширяет Iterable<E>, поэтому по любой коллекции можно пройти циклом for-each и получить Iterator.

Интерфейс java.util.Map<K, V> описывает отображение ключ → значение. Ключи уникальны; по ключу выполняют поиск, вставку и удаление. Map не является наследником Collection (это другая абстракция), но набор ключей, значений или пар “ключ-значение” можно получить представлениями keySet(), values(), entrySet().







Рисунки 14-16. Диаграммы описывающие Collection и Map

Таблица 1. Основные методы интерфейса Collection

Метод	**Описание**	**Пример использования**
add(E e)	Добавляет элемент в коллекцию. Возвращает true, если коллекция изменилась.	collection.add("Apple");
remove(Object o)	Удаляет указанный элемент из коллекции, если он там есть.	collection.remove("Apple");
contains(Object o)	Проверяет, содержится ли элемент в коллекции (возвращает true или false).	if (collection.contains("Apple"))
size()	Возвращает количество элементов в коллекции.	int count = collection.size();
isEmpty()	Проверяет, пуста ли коллекция.	if (collection.isEmpty())

Метод	**Описание**	**Пример использования**
clear()	Полностью очищает коллекцию, удаляя все элементы.	collection.clear();
iterator()	Возвращает итератор для обхода элементов коллекции.	Iterator<String> it = collection.iterator();
toArray()	Преобразует коллекцию в обычный массив объектов.	Object[] arr = collection.toArray();

Таблица 2. Основные методы интерфейса Map

Метод	**Описание**	**Пример использования**
put(K key, V value)	Добавляет пару "ключ-значение". Если ключ уже есть, заменяет старое значение.	map.put("id_123", "John");

get(Object key)	Возвращает значение по указанному ключу (или null, если ключа нет).	String name = map.get("id_123");
remove(Object key)	Удаляет из мапы пару по указанному ключу.	map.remove("id_123");
containsKey(Object key)	Проверяет, есть ли такой ключ в мапе.	if (map.containsKey("id_123"))
containsValue(Object v)	Проверяет, есть ли такое значение (хотя бы одно) в мапе.	if (map.containsValue("John"))
keySet()	Возвращает множество (Set) всех ключей, содержащихся в мапе.	Set<String> keys = map.keySet();
values()	Возвращает коллекцию (Collection) всех значений из мапы.	Collection<String> vals = map.values();
entrySet()	Возвращает множество пар (Set<Map.Entry>) для итерации по ключам и значениям одновременно.	for (Map.Entry e : map.entrySet())

2.5 Приведите описание реализации конкретной коллекции (как работает механизм добавления элементов, как работает механизм удаления элементов, как работает механизм поиска элементов и т.д.)

Напишите конкретную вычислительную сложность добавления, удаления и получения элементов в каждой коллекции в соответствии с вариантом;

Каким образом хранятся элементы в коллекции? (упорядочено или хаотично и т.п.)

ArrayList — это одна из самых популярных структур данных в Java (и других языках), которая представляет собой динамический массив. В отличие от обычного массива, его размер может изменяться во время выполнения программы.

2.5.1 Внутреннее устройство и хранение элементов

В основе ArrayList лежит обычный массив объектов (в Java это Object[] elementData).

- Как хранятся элементы: Элементы хранятся упорядоченно (в порядке их добавления). Каждому элементу присваивается индекс, начиная с 0.
- Память: Элементы располагаются в памяти вплотную друг к другу в непрерывном блоке, что обеспечивает высокую скорость доступа.

2.5.2 Механизмы работы (Алгоритмы)

Добавление элементов (add)

Когда вы добавляете элемент, происходит следующее:

1. Проверка вместимости: Проверяется, есть ли в текущем массиве свободное место.
2. Расширение (Resizing): Если массив заполнен, создается новый массив большего размера (обычно в 1.5 раза больше предыдущего: $newCapacity = oldCapacity + (oldCapacity >> 1)$).
3. Копирование: Все элементы из старого массива копируются в новый с помощью System.arraycopy().
4. Вставка: Новый элемент записывается в первую свободную ячейку.

Удаление элементов (remove)

Удаление — более «дорогая» операция, чем добавление в конец:

1. Поиск индекса: Находится индекс удаляемого элемента.
2. Сдвиг: Все элементы, находящиеся справа от удаляемого, сдвигаются на одну позицию влево, чтобы закрыть «дыру».
3. Очистка: Последняя (теперь лишняя) ячейка зануляется (null), чтобы помочь сборщику мусора.

Поиск элементов (get и indexOf)

- По индексу (get): Прямое обращение к ячейке массива. Это происходит мгновенно.
- По значению (indexOf): Происходит перебор массива в цикле (линейный поиск) до тех пор, пока не будет найдено совпадение через equals().

Таблица 3. Вычислительная сложность (Big O)

Операция	Сложность (Big O)	Комментарий
Добавление в конец	$O(1)$*	Константное время, но $O(n)$ при расширении массива (амортизированное $O(1)$).
Добавление в середину/начало	$O(n)$	Требуется сдвиг всех элементов справа от места вставки.
Удаление (по индексу)	$O(n)$	Требуется сдвиг элементов влево.

Получение (по индексу)	$O(1)$	Самая сильная сторона: мгновенный доступ.
Поиск (по значению)	$O(n)$	В худшем случае нужно проверить все элементы.

ArrayList идеален, если нужно часто читать данные по индексу и вы преимущественно добавляете элементы в конец списка. Однако, если задача подразумевает постоянное удаление или вставку элементов в середину/начало списка, лучше рассмотреть LinkedList, так как ArrayList тратит много ресурсов на постоянное копирование и сдвиг элементов.

2.6 Необходимо произвести вычисление времени работы нижеперечисленных операций, количество элементов коллекциях и сами коллекции указаны в индивидуальном варианте. После выполнения вычислений необходимо заполнить таблицы и привести скриншоты кода программы и данных вывода в консоль.

1. Сравнение времени выполнения операции добавление элемента в начало коллекции;
2. Сравнение времени выполнения операции добавление элемента в конец коллекции;
3. Сравнение времени выполнения операции добавление элемента в середину коллекции;
4. Сравнение времени выполнения операции удаления элемента в начале коллекции;
5. Сравнение времени выполнения операции удаления элемента в конце коллекции;
6. Сравнение времени выполнения операции удаления элемента в середине коллекции;
7. Сравнение времени выполнения операции получения элемента по индексу;

Таблица 4. Сравнение времени выполнения операции добавления

Коллекция	в начало коллекции	в середину коллекции	в конец коллекции
ArrayList	361	190	3
HashMap	6	10	5
LinkedList	3	3697	3

Таблица 5. Сравнение времени выполнения операции удаления

Коллекция	в начале коллекции	в середине коллекции	в конце коллекции
ArrayList	341	175	3
HashMap	4	41249	1
LinkedList	4	3111	2

Таблица 6. Сравнение времени выполнения операции получения элемента по индексу

Коллекция	По индексу для <n> элементов
ArrayList	1
HashMap	2
LinkedList	3268

2.7 Дайте определение понятию “итератор”.

Итератор - это объект, который предоставляет стандартный интерфейс для последовательного обхода элементов коллекции (списка, множества, карты и т. д.), не раскрывая при этом ее внутреннего устройства.

Вывод:

В ходе выполнения лабораторной работы были изучены динамические структуры и коллекции Java. Установлено: Рекурсия удобна для вложенных задач, но есть стек ArrayList идеален для чтения, но медленен при частых изменениях в середине. Итераторы обеспечивают универсальный и безопасный способ обхода данных.Выбор коллекции зависит от баланса операций чтение притив запись.

n0vision/Java_Labs_2026